first

February 28, 2026

1 Projektovanje konvolucione neuralne mreže za klasifikaciju slika

1.1 AI vs Human Generated Images

Opis problema: Klasifikacija slika u dve klase — slike generisane veštačkom inteligencijom (AI) i autentične slike koje su nastale fotografisanjem (Human). Model treba da nauči da prepoznaje razlike između ove dve kategorije na osnovu vizuelnih karakteristika.

1.2 1. Imports

[1]:
```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
import os

from keras.utils import image_dataset_from_directory
from keras.models import Sequential
from keras import layers
from keras.callbacks import EarlyStopping
from keras.losses import SparseCategoricalCrossentropy

from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.metrics import classification_report

print(f'TensorFlow version: {tf.__version__}')
print(f'GPU available: {len(tf.config.list_physical_devices("GPU")) > 0}')
```

```
2026-02-27 22:59:18.001468: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:31]
Could not find cuda drivers on your machine, GPU will not be used.
2026-02-27 22:59:18.001902: I tensorflow/core/util/port.cc:153] oneDNN custom
operations are on. You may see slightly different numerical results due to
floating-point round-off errors from different computation orders. To turn them
off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-02-27 22:59:18.061369: I tensorflow/core/platform/cpu_feature_guard.cc:210]
This TensorFlow binary is optimized to use available CPU instructions in
performance-critical operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16 FMA,
```

```
in other operations, rebuild TensorFlow with the appropriate compiler flags.
2026-02-27 22:59:19.507274: I tensorflow/core/util/port.cc:153] oneDNN custom
operations are on. You may see slightly different numerical results due to
floating-point round-off errors from different computation orders. To turn them
off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-02-27 22:59:19.508327: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:31]
Could not find cuda drivers on your machine, GPU will not be used.

TensorFlow version: 2.20.0
GPU available: False

2026-02-27 22:59:19.906369: E
external/local_xla/xla/stream_executor/cuda/cuda_platform.cc:51] failed call to
cuInit: INTERNAL: CUDA error: Failed call to cuInit: UNKNOWN ERROR (303)
```

1.3 2. Preuzimanje skupa podataka sa Kaggle

[2]:
```python
import subprocess
import os

# Postavljanje Kaggle API tokena
os.environ['KAGGLE_API_TOKEN'] = 'KGAT_fd2c5151f762387148d28c5a8dd2d08f'

DATA_DIR = './data'

if not os.path.exists(os.path.join(DATA_DIR, 'train')):
    print('Preuzimanje dataseta...')
    subprocess.run([
        'kaggle', 'datasets', 'download',
        '-d', 'alessandrasala79/ai-vs-human-generated-dataset',
        '-p', DATA_DIR, '--unzip'
    ], check=True)
    print('Dataset preuzet i raspakovan.')
else:
    print('Dataset vec postoji.')
```

```
Preuzimanje dataseta…
Dataset URL: https://www.kaggle.com/datasets/alessandrasala79/ai-vs-human-
generated-dataset
License(s): apache-2.0
User cancelled operation
```

```
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
Cell In[2], line 11
      9 if not os.path.exists(os.path.join(DATA_DIR, 'train')):
     10     print('Preuzimanje dataseta…')
---> 11     subprocess.run([
```

```
     12         'kaggle', 'datasets', 'download',
     13         '-d', 'alessandrasala79/ai-vs-human-generated-dataset',
     14         '-p', DATA_DIR, '--unzip'
     15     ], check=True)
     16     print('Dataset preuzet i raspakovan.')
     17 else:

File /usr/lib/python3.10/subprocess.py:505, in run(input, capture_output,␣
 ↪timeout, check, *popenargs, **kwargs)
    503 with Popen(*popenargs, **kwargs) as process:
    504     try:
--> 505         stdout, stderr = process.communicate(input, timeout=timeout)
    506     except TimeoutExpired as exc:
    507         process.kill()

File /usr/lib/python3.10/subprocess.py:1146, in Popen.communicate(self, input,␣
 ↪timeout)
   1144         stderr = self.stderr.read()
   1145         self.stderr.close()
-> 1146     self.wait()
   1147 else:
   1148     if timeout is not None:

File /usr/lib/python3.10/subprocess.py:1209, in Popen.wait(self, timeout)
   1207     endtime = _time() + timeout
   1208 try:
-> 1209     return self._wait(timeout=timeout)
   1210 except KeyboardInterrupt:
   1211     # https://bugs.python.org/issue25942
   1212     # The first keyboard interrupt waits briefly for the child to
   1213     # exit under the common assumption that it also received the ^C
   1214     # generated SIGINT and will exit rapidly.
   1215     if timeout is not None:

File /usr/lib/python3.10/subprocess.py:1959, in Popen._wait(self, timeout)
   1957 if self.returncode is not None:
   1958     break  # Another thread waited.
-> 1959 (pid, sts) = self._try_wait(0)
   1960 # Check the pid and loop as waitpid has been known to
   1961 # return 0 even without WNOHANG in odd situations.
   1962 # http://bugs.python.org/issue14396.
   1963 if pid == self.pid:

File /usr/lib/python3.10/subprocess.py:1917, in Popen._try_wait(self, wait_flag
   1915 """All callers to this function MUST hold self._waitpid_lock."""
   1916 try:
-> 1917     (pid, sts) = os.waitpid(self.pid, wait_flags)
   1918 except ChildProcessError:
```

```
   1919     # This happens if SIGCLD is set to be ignored or waiting
   1920     # for child processes has otherwise been disabled for our
   1921     # process.  This child is dead, we can't get the status.
   1922     pid = self.pid

KeyboardInterrupt:
```

[3]:
```
# Pregled strukture foldera
for root, dirs, files in os.walk(DATA_DIR):
    depth = root.replace(DATA_DIR, '').count(os.sep)
    indent = ' ' * 2 * depth
    print(f'{indent}{os.path.basename(root)}/')
    if depth < 2:
        for d in dirs:
            subdir_path = os.path.join(root, d)
            n_files = len([f for f in os.listdir(subdir_path) if os.path.
 ↪isfile(os.path.join(subdir_path, f))])
            print(f'{indent}  {d}/ ({n_files} fajlova)')
```

```
data/
  train_data/ (79950 fajlova)
  test_data_v2/ (5540 fajlova)
  train_data/
  test_data_v2/
```

1.4 3. Učitavanje i eksploracija podataka

[4]:
```
# Parametri
IMG_SIZE = (128, 128)
BATCH_SIZE = 64
SEED = 42

# Proveriti tačnu putanju nakon preuzimanja dataseta.
# Prilagoditi TRAIN_DIR i TEST_DIR prema strukturi raspakovanih fajlova.
TRAIN_DIR = os.path.join(DATA_DIR, 'train')
TEST_DIR = os.path.join(DATA_DIR, 'test')

# Ako dataset nema poseban test folder, koristimo validation_split
# Za slucaj da postoji samo jedan folder sa klasama:
if os.path.exists(TRAIN_DIR):
    # Train set delimo na train i validaciju
    Xtrain, Xval = image_dataset_from_directory(
        TRAIN_DIR,
        subset='both',
        validation_split=0.2,
        image_size=IMG_SIZE,
        batch_size=BATCH_SIZE,
```

```
        shuffle=True,
        seed=SEED
    )

    # Test set
    if os.path.exists(TEST_DIR):
        Xtest = image_dataset_from_directory(
            TEST_DIR,
            image_size=IMG_SIZE,
            batch_size=BATCH_SIZE,
            shuffle=False
        )
    else:
        print('Test folder ne postoji - koristimo validacioni skup kao test.')
        Xtest = Xval
else:
    # Ako je sve u jednom folderu
    main_dir = DATA_DIR
    Xtrain, Xval = image_dataset_from_directory(
        main_dir,
        subset='both',
        validation_split=0.3,
        image_size=IMG_SIZE,
        batch_size=BATCH_SIZE,
        shuffle=True,
        seed=SEED
    )
    Xtest = Xval

classes = Xtrain.class_names
num_classes = len(classes)
print(f'Klase: {classes}')
print(f'Broj klasa: {num_classes}')
```

```
Found 85490 files belonging to 2 classes.
Using 59843 files for training.
Using 25647 files for validation.
Klase: ['test_data_v2', 'train_data']
Broj klasa: 2
```

[5]:
```
# Prikaz jednog primerka iz svake klase
fig, axes = plt.subplots(1, num_classes, figsize=(5 * num_classes, 5))
if num_classes == 1:
    axes = [axes]

shown_classes = set()
for images, labels in Xtrain:
```

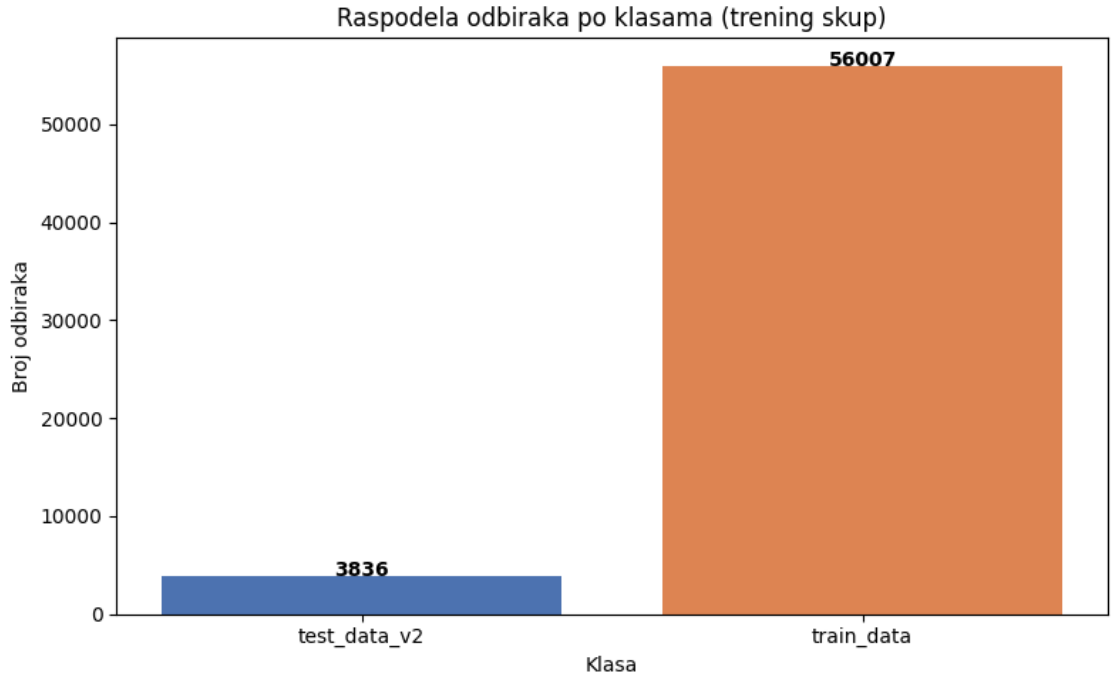
```
    for i in range(len(labels)):
        label = labels[i].numpy()
        if label not in shown_classes:
            ax = axes[label]
            ax.imshow(images[i].numpy().astype('uint8'))
            ax.set_title(classes[label], fontsize=14)
            ax.axis('off')
            shown_classes.add(label)
        if len(shown_classes) == num_classes:
            break
    if len(shown_classes) == num_classes:
        break

plt.suptitle('Jedan primerak iz svake klase', fontsize=16)
plt.tight_layout()
plt.show()
```

[6]:
```
# Histogram raspodele odbiraka po klasama
class_counts = {c: 0 for c in classes}

for _, labels in Xtrain.unbatch():
    class_counts[classes[labels.numpy()]] += 1

print('Broj odbiraka po klasama (trening skup):')
for c, count in class_counts.items():
    print(f'  {c}: {count}')
```

```
plt.figure(figsize=(8, 5))
plt.bar(class_counts.keys(), class_counts.values(), color=['#4C72B0',␣
 ↪'#DD8452'][:num_classes])
plt.xlabel('Klasa')
plt.ylabel('Broj odbiraka')
plt.title('Raspodela odbiraka po klasama (trening skup)')
for i, (c, v) in enumerate(class_counts.items()):
    plt.text(i, v + 10, str(v), ha='center', fontweight='bold')
plt.tight_layout()
plt.show()

# Provera balansiranosti
counts = list(class_counts.values())
ratio = max(counts) / min(counts) if min(counts) > 0 else float('inf')
print(f'\nOdnos najvece/najmanje klase: {ratio:.2f}')
if ratio < 1.5:
    print('Podaci su priblizno balansirani.')
else:
    print('Podaci NISU balansirani - potrebno je primeniti tehniku balansiranja.
 ↪')
```

```
Broj odbiraka po klasama (trening skup):
  test_data_v2: 3836
  train_data: 56007
```

```
2026-02-27 23:00:11.953772: I tensorflow/core/framework/local_rendezvous.cc:407]
Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
```



```
Odnos najvece/najmanje klase: 14.60
Podaci NISU balansirani - potrebno je primeniti tehniku balansiranja.
```

[7]:
```
# Racunanje class_weight u slucaju nebalansiranih klasa
from sklearn.utils.class_weight import compute_class_weight

all_labels = []
for _, labels in Xtrain.unbatch():
    all_labels.append(labels.numpy())
all_labels = np.array(all_labels)

weights = compute_class_weight('balanced', classes=np.unique(all_labels),␣
 ↪y=all_labels)
class_weight = dict(enumerate(weights))
print(f'Class weights: {class_weight}')
```

```
Class weights: {0: np.float64(7.800182481751825), 1:
np.float64(0.5342457192850893)}
```

```
2026-02-27 23:00:29.691620: I tensorflow/core/framework/local_rendezvous.cc:407]
Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
```

1.5 4. Predprocesiranje i augmentacija podataka

Primenjene transformacije: - **Skaliranje (Rescaling):** Vrednosti piksela se normalizuju sa opsega [0, 255] na [0, 1] kako bi se ubrzala konvergencija treninga. - **Horizontalno okretanje (RandomFlip):** Slika se nasumično okreće po horizontalnoj osi, čime se povećava raznovrsnost trening skupa. - **Nasumična rotacija (RandomRotation):** Slika se rotira za nasumični ugao, što pomaže modelu da bude otporniji na orijentaciju objekata. - **Nasumični zum (RandomZoom):** Slika se nasumično zumira, simulirajući različite udaljenosti kamere.

Ove augmentacije se primenjuju samo tokom treninga i pomažu u smanjenju preobucenosti (overfitting).

[8]:
```
# Augmentacija podataka
data_augmentation = Sequential([
    layers.RandomFlip('horizontal'),
    layers.RandomRotation(0.15),
    layers.RandomZoom(0.1),
])

# Prikaz augmentiranih slika
plt.figure(figsize=(12, 6))
for images, _ in Xtrain.take(1):
    img = images[0]
```

```
    for i in range(8):
        aug_img = data_augmentation(tf.expand_dims(img, 0))
        plt.subplot(2, 4, i + 1)
        plt.imshow(aug_img[0].numpy().astype('uint8'))
        plt.axis('off')
plt.suptitle('Primeri augmentacije jedne slike', fontsize=14)
plt.tight_layout()
plt.show()
```

1.6 5. Definisanje CNN modela

Projektovana je konvoluciona neuralma mreža sa 4 konvoluciona bloka.

Obrazloženje izbora: - **Kriterijumska funkcija:** `SparseCategoricalCrossentropy` — standardni izbor za klasifikaciju u više klasa gde su labele celobrojne. - **Funkcija aktivacije neurona:** `ReLU` u skrivenim slojevima (rešava problem nestajućeg gradijenta, računski efikasna), `Softmax` u izlaznom sloju (daje distribuciju verovatnoća po klasama). - **Optimizacija:** `Adam` — adaptivna metoda optimizacije koja kombinuje prednosti AdaGrad i RMSProp algoritama, dobro funkcioniše sa podrazumevanim hiperparametrima.

Zaštita od preobučavanja (overfitting):

Preobučavanje je pojava kada model previše dobro nauči trening podatke, uključujući šum i specifičnosti, pa ne uspeva da generalizuje na nove, neviđene podatke. To se manifestuje kroz veliku razliku između performansi na trening i validacionom/test skupu.

Primenjene metode zaštite: - **Dropout (0.3):** Tokom treninga, nasumično isključuje 30% neurona, čime se sprečava ko-adaptacija i forsira redundantna reprezentacija. - **Batch Normalization:** Normalizuje aktivacije unutar svakog mini-batch-a, stabilizuje trening i ima blagi regularizacioni efekat. - **Early Stopping:** Prekida trening kada se validaciona metrika prestane poboljšavati,

čime se sprečava nepotrebno treniranje koje vodi ka preobučavanju.

[9]:
```
def cnn_model(num_classes):
    model = Sequential([
        # Augmentacija i skaliranje
        data_augmentation,
        layers.Rescaling(1./255),

        # Konvolucioni blok 1
        layers.Conv2D(32, 3, padding='same', activation='relu'),
        layers.BatchNormalization(),
        layers.MaxPooling2D(),

        # Konvolucioni blok 2
        layers.Conv2D(64, 3, padding='same', activation='relu'),
        layers.BatchNormalization(),
        layers.MaxPooling2D(),

        # Konvolucioni blok 3
        layers.Conv2D(128, 3, padding='same', activation='relu'),
        layers.BatchNormalization(),
        layers.MaxPooling2D(),

        # Konvolucioni blok 4
        layers.Conv2D(256, 3, padding='same', activation='relu'),
        layers.BatchNormalization(),
        layers.MaxPooling2D(),

        # Klasifikaciona glava
        layers.Dropout(0.3),
        layers.Flatten(),
        layers.Dense(128, activation='relu'),
        layers.Dropout(0.3),
        layers.Dense(num_classes, activation='softmax')
    ])

    model.compile(
        optimizer='adam',
        loss=SparseCategoricalCrossentropy(),
        metrics=['accuracy']
    )

    return model
```

[10]:
```
model = cnn_model(num_classes)
model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
sequential (Sequential)	(1, 128, 128, 3)	0
rescaling (Rescaling)	(1, 128, 128, 3)	0
conv2d (Conv2D)	(1, 128, 128, 32)	896
batch_normalization (BatchNormalization)	(1, 128, 128, 32)	128
max_pooling2d (MaxPooling2D)	(1, 64, 64, 32)	0
conv2d_1 (Conv2D)	(1, 64, 64, 64)	18,496
batch_normalization_1 (BatchNormalization)	(1, 64, 64, 64)	256
max_pooling2d_1 (MaxPooling2D)	(1, 32, 32, 64)	0
conv2d_2 (Conv2D)	(1, 32, 32, 128)	73,856
batch_normalization_2 (BatchNormalization)	(1, 32, 32, 128)	512
max_pooling2d_2 (MaxPooling2D)	(1, 16, 16, 128)	0
conv2d_3 (Conv2D)	(1, 16, 16, 256)	295,168
batch_normalization_3 (BatchNormalization)	(1, 16, 16, 256)	1,024
max_pooling2d_3 (MaxPooling2D)	(1, 8, 8, 256)	0
dropout (Dropout)	(1, 8, 8, 256)	0
flatten (Flatten)	(1, 16384)	0
dense (Dense)	(1, 128)	2,097,280
dropout_1 (Dropout)	(1, 128)	0
dense_1 (Dense)	(1, 2)	258

```
Total params: 2,487,874 (9.49 MB)

Trainable params: 2,486,914 (9.49 MB)

Non-trainable params: 960 (3.75 KB)
```

1.7 6. Treniranje modela

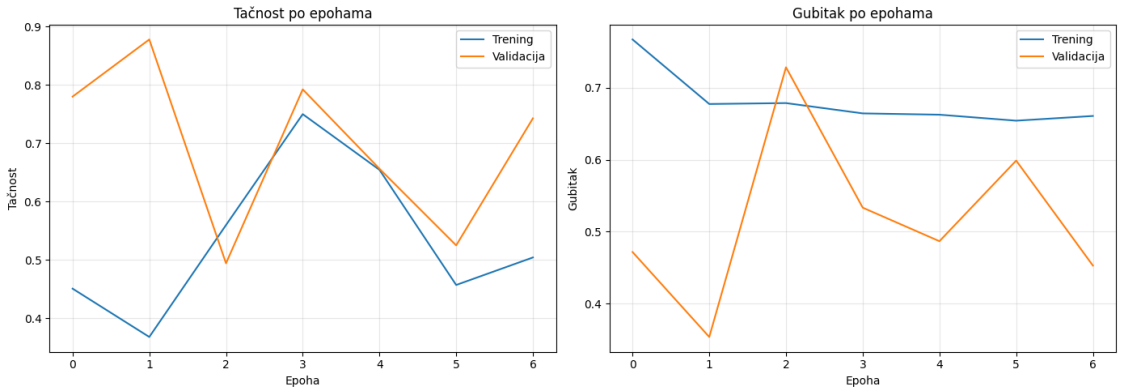
[11]:
```
es = EarlyStopping(
    monitor='val_accuracy',
    patience=5,
    restore_best_weights=True,
    verbose=1
)

history = model.fit(
    Xtrain,
    epochs=30,
    validation_data=Xval,
    callbacks=[es],
    class_weight=class_weight,
    verbose=1
)
```

```
Epoch 1/30
936/936             277s 293ms/step -
accuracy: 0.4506 - loss: 0.7671 - val_accuracy: 0.7797 - val_loss: 0.4716
Epoch 2/30
936/936             342s 365ms/step -
accuracy: 0.3677 - loss: 0.6774 - val_accuracy: 0.8777 - val_loss: 0.3535
Epoch 3/30
936/936             350s 374ms/step -
accuracy: 0.5596 - loss: 0.6787 - val_accuracy: 0.4940 - val_loss: 0.7285
Epoch 4/30
936/936             351s 375ms/step -
accuracy: 0.7497 - loss: 0.6644 - val_accuracy: 0.7922 - val_loss: 0.5333
Epoch 5/30
936/936             351s 375ms/step -
accuracy: 0.6543 - loss: 0.6626 - val_accuracy: 0.6561 - val_loss: 0.4867
Epoch 6/30
936/936             350s 374ms/step -
accuracy: 0.4568 - loss: 0.6542 - val_accuracy: 0.5247 - val_loss: 0.5988
Epoch 7/30
936/936             349s 372ms/step -
accuracy: 0.5040 - loss: 0.6608 - val_accuracy: 0.7425 - val_loss: 0.4528
Epoch 7: early stopping
```

```
Restoring model weights from the end of the best epoch: 2.
```

1.8 7. Grafik promena performansi tokom epoha treniranja

[12]:
```python
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Tacnost
ax1.plot(history.history['accuracy'], label='Trening')
ax1.plot(history.history['val_accuracy'], label='Validacija')
ax1.set_title('Tačnost po epohama')
ax1.set_xlabel('Epoha')
ax1.set_ylabel('Tačnost')
ax1.legend()
ax1.grid(True, alpha=0.3)

# Gubitak
ax2.plot(history.history['loss'], label='Trening')
ax2.plot(history.history['val_loss'], label='Validacija')
ax2.set_title('Gubitak po epohama')
ax2.set_xlabel('Epoha')
ax2.set_ylabel('Gubitak')
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



1.9 8. Evaluacija modela

[13]:
```python
# Predikcije na test skupu
y_true_test = np.array([])
y_pred_test = np.array([])
for img, lab in Xtest:
```

```
    y_true_test = np.concatenate([y_true_test, lab.numpy()])
    y_pred_test = np.concatenate([y_pred_test, np.argmax(model.predict(img,␣
 ↪verbose=0), axis=1)])

print(f'Tačnost modela na test skupu: {100 * accuracy_score(y_true_test,␣
 ↪y_pred_test):.2f}%')
print()
print('Detaljan izveštaj:')
print(classification_report(y_true_test, y_pred_test, target_names=classes))
```

```
Tačnost modela na test skupu: 87.77%

Detaljan izveštaj:
              precision    recall  f1-score   support

test_data_v2       0.09      0.09      0.09      1704
  train_data       0.94      0.93      0.93     23943

    accuracy                           0.88     25647
   macro avg       0.51      0.51      0.51     25647
weighted avg       0.88      0.88      0.88     25647


2026-02-28 00:03:21.884341: I tensorflow/core/framework/local_rendezvous.cc:407]
Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
```

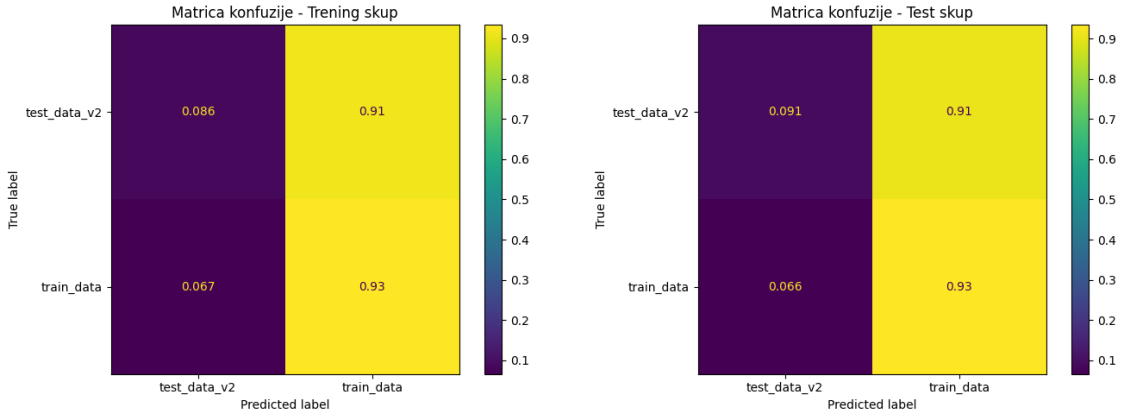
[14]:
```
# Predikcije na trening skupu
y_true_train = np.array([])
y_pred_train = np.array([])
for img, lab in Xtrain:
    y_true_train = np.concatenate([y_true_train, lab.numpy()])
    y_pred_train = np.concatenate([y_pred_train, np.argmax(model.predict(img,␣
 ↪verbose=0), axis=1)])

print(f'Tačnost modela na trening skupu: {100 * accuracy_score(y_true_train,␣
 ↪y_pred_train):.2f}%')
```

```
Tačnost modela na trening skupu: 87.91%
```

1.10 9. Matrica konfuzije

[15]:
```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Matrica konfuzije - trening skup
cm_train = confusion_matrix(y_true_train, y_pred_train, normalize='true')
ConfusionMatrixDisplay(confusion_matrix=cm_train, display_labels=classes).
 ↪plot(ax=ax1)
```

```
ax1.set_title('Matrica konfuzije - Trening skup')

# Matrica konfuzije - test skup
cm_test = confusion_matrix(y_true_test, y_pred_test, normalize='true')
ConfusionMatrixDisplay(confusion_matrix=cm_test, display_labels=classes).
 ↪plot(ax=ax2)
ax2.set_title('Matrica konfuzije - Test skup')

plt.tight_layout()
plt.show()
```



1.11 10. Primeri dobro i loše klasifikovanih slika

[16]:
```
# Prikupljanje slika, pravih labela i predikcija sa test skupa
all_images = []
all_true = []
all_pred = []

for img_batch, lab_batch in Xtest:
    preds = np.argmax(model.predict(img_batch, verbose=0), axis=1)
    for i in range(len(lab_batch)):
        all_images.append(img_batch[i].numpy().astype('uint8'))
        all_true.append(lab_batch[i].numpy())
        all_pred.append(preds[i])

all_true = np.array(all_true)
all_pred = np.array(all_pred)

correct_idx = np.where(all_true == all_pred)[0]
incorrect_idx = np.where(all_true != all_pred)[0]

print(f'Ukupno tačno klasifikovanih: {len(correct_idx)}')
```

```
print(f'Ukupno pogrešno klasifikovanih: {len(incorrect_idx)}')
```

```
Ukupno tačno klasifikovanih: 22511
Ukupno pogrešno klasifikovanih: 3136
```

[17]:
```
# Dobro klasifikovane slike
n_show = min(8, len(correct_idx))
fig, axes = plt.subplots(2, 4, figsize=(16, 8))
sample = np.random.choice(correct_idx, n_show, replace=False)
for i, idx in enumerate(sample):
    ax = axes[i // 4][i % 4]
    ax.imshow(all_images[idx])
    ax.set_title(f'Stvarno: {classes[all_true[idx]]}\nPredikcija: ␣
 ↪{classes[all_pred[idx]]}', fontsize=10)
    ax.axis('off')
plt.suptitle('Primeri DOBRO klasifikovanih slika', fontsize=14)
plt.tight_layout()
plt.show()
```

[18]:
```
# Lose klasifikovane slike
n_show = min(8, len(incorrect_idx))
if n_show > 0:
    fig, axes = plt.subplots(2, 4, figsize=(16, 8))
    sample = np.random.choice(incorrect_idx, n_show, replace=False)
    for i, idx in enumerate(sample):
        ax = axes[i // 4][i % 4]
        ax.imshow(all_images[idx])
```

```
        ax.set_title(f'Stvarno: {classes[all_true[idx]]}\nPredikcija:␣
↪{classes[all_pred[idx]]}', fontsize=10, color='red')
        ax.axis('off')
    # Sakrij prazne subplotove
    for i in range(n_show, 8):
        axes[i // 4][i % 4].axis('off')
    plt.suptitle('Primeri POGREŠNO klasifikovanih slika', fontsize=14)
    plt.tight_layout()
    plt.show()
else:
    print('Nema pogrešno klasifikovanih slika!')
```